

INSY7315

WORK INTEGRATED LEARNING

TCMNL SOLUTIONS

MATTHEW BOWES	ST10444262
LEO ADDINAL	ST10441154
CONNOR MCCLURE	ST10435598
NAFEES SAMPSON	ST10456389
THOMAS NEWMAN	ST10456111

Company – Inflamm

The logo for Inflamm, featuring the word "Inflamm" in a bold, red, sans-serif font. The letter "l" is stylized with a red flame-like shape rising from its top.

Table of Contents

Table of Figures	4
Introduction	5
Requirements	6
Introduction	6
Organisation Research & Background	6
User Roles	8
Customer	8
Staff	8
Admin	8
User Stories	9
Customer	9
Staff	9
Admin	10
User Experience Journey Map	11
Work Breakdown Structure	13
Non-Functional Requirements	24
Performance	24
Scalability	24
Availability	25
Reliability	25
Security	26
Maintainability	26
Analysis Artifacts	28
Domain Modelling	28
Description:	28
Description:	29
Implementation	31
Description:	31
Description	32
Deployment Plan	35
Platform	35
Environment Setup	35
Rollout Strategy	35
Rationale	36
Data Schema	37
Primary data storage:	37
Caching: Redis via Upstash	38

File/Object Storage: Supabase Storage	38
Scalability	38
Architecture Artifacts	40
Design Patterns	40
Singleton Pattern (Creational)	40
Factory Method Pattern (Creational)	40
Builder Pattern (Creational)	40
Adapter Pattern (Structural)	41
Decorator Pattern (Structural)	41
Facade Pattern (Structural)	41
Command Pattern (Behavioural)	41
Mediator Pattern (Behavioural)	42
Observer Pattern (Behavioural)	42
State Pattern (Behavioural)	42
Chain of Responsibility Pattern (Behavioural)	43
UML Diagram	44
Architecture Patterns	45
Layered (N-Tier) Pattern	45
CQRS (Command Query Responsibility Segregation) Pattern	45
Repository Pattern	45
Client-Server Pattern	45
System Architecture Diagram	46
Cloud	47
Network Segregation	48
Protocols between Services	49
Security	50
Cross-Site Scripting (XSS) Attacks	50
Risk	50
Mitigation Strategy	50
Unauthorized Access and Privileged Escalation	50
Risk	50
Mitigation	51
Distributed Denial of Service (DDoS)	51
Risk	51
Mitigation	51
Data Interception (Man-in-the-Middle)	52
Risk	52
Mitigation	52
Brute Force and Credential Stuffing	52

Risk.....	52
Mitigation	52
Authentication and Authorization (Admin Portal).....	53
Network Security and Perimeter Defence	53
Architectural Security	54
DevOps	55
Version Control and Branching Strategy	55
Branch Hierarchy.....	56
Collaboration and Pull Requests (PR) Workflow	57
Isolation	57
Pull Request Submission	57
Code Review	57
Automated Test and Quality Assurance	57
Backend Testing (C# .NET Core API).....	57
Frontend Testing (Next.js).....	57
Automated Quality Gate	58
GitHub Actions Pipeline	58
Phase 1: CI – Triggered on PR to “develop” or “main”	58
Phase 2: CD – Triggered on Merge to “main”	58
Running Costs	60
Predicted Monthly Running Cost	60
Monthly Cost of service	60
Cost at scale	61
Two-Year Growth Projection.....	62
Change Management	63
How and why will the organization implement the software?	63
How and why will the users adopt the software?.....	63
Strategy to gain adoption from both the organisation and users.....	64
Handover and Transition plan.....	65
Success Indicators.....	66
Contingency Plan.....	66
Prototype	68
Declaration of authenticity	69
References	72

Table of Figures

Figure 1: Public Website UX Journey Map	11
Figure 2: Admin UX Journey Map	12
Figure 3: UML Domain Class Diagram	28
Figure 4: UML Design Class Diagram	29
Figure 5: UML Sequence diagram : Client Submits an Enquiry	31
Figure 6: UML Sequence Diagram: Staff Reviews and Decides on an Enquiry	32
Figure 7: UML State Diagram: Enquiry Status Lifecycle	33
Figure 8: Deployment Diagram: System Hosting on Render	34
Figure 9: Entity Relationship Diagram	39
Figure 10: UML Diagram for Design Patterns	44
Figure 11: System Architecture Diagram	46
Figure 12: Cloud Architecture Diagram	48
Figure 13: GitHub Flows Branch Structure	56
Figure 14: GitHub Actions Pipeline Diagram	59

Introduction

Inflame is a company that specializes in the design, supply, and installation of fireplaces and braais. In today's evolving world, digitalization has become a very important factor in the market environment. Consumers expect to be able to research products, find out their prices and make purchases all in one go without having to wait for long hours for their phone call to be answered or visit the store physically. Given the company's excellent reputation for high-quality fireplace installations, extending that same level of service through an internet-based system presents a valuable opportunity to strengthen customer satisfaction and support continued business growth. In this document, we suggest developing an internet-based system that will allow customers to make requests for getting the estimated quote and have the option of completing their order without going through any unnecessary checkups, creating a seamless experience for the consumer. The sections that follow will look at the system's requirements, design, and implementation plan, along with the strategy for managing its adoption and rollout within the organization.

Requirements

Introduction

This section will outline the research and requirement artifacts that serve as the foundation for what will be developed. This is done to ensure the system developed will adequately satisfy the needs of Inflamm and all associated stakeholders. The knowledge and understanding gained during our interactions with Inflamm have formed the basis of the user roles and stories as well as the UX journey map, work breakdown structure and non-functional requirements that are to follow.

Organisation Research & Background

Inflamm is a family-owned small business located in the Southern Suburbs in Cape Town. They serve as a light engineering company, specializing in the production and installation of fireplaces and braais throughout Cape Town and surrounding areas. They currently operate without a digital footprint as they conduct business without a website to advertise their services or to help provide a consistent influx of new and returning customers. All paperwork, management of orders as well as invoicing is currently being completed manually by the owner within either a word document or a dedicated hardcover book. This has led to the business losing out to competitors on potential clients as well immense mental strain for the owner when trying to manage all the logistics of the company without the aid of modern technology.

TCMNL solutions conducted extensive interviews with the owner of Inflamm while analysing solutions by competitors with the owner to determine exactly what needs to be done to help bridge the gap in technology his company is currently facing. The conclusion both parties arrived at is TCMNL solutions will provide Inflamm with two distinct software solutions, firstly a website advertising Inflamm's current catalogue with an estimated quote generator and room size calculator where clients can enter the measurements of their room to receive an estimated ideal fireplace size based on the room's dimensions with the solution also allowing clients to either reach out to the owner personally or clients can leave their details and allow the owner to contact them.

Secondly TCMNL solutions will provide Inflamm with a separate dashboard application which will handle all the logistics of the company such as storing invoices, paperwork, keeping a history of orders and clients, reducing mental strain on the owner and allowing him to fully focus on successfully running and growing the business.

User Roles

User roles that were identified for the system:

Customer

A member of the public who browses the catalogue and gallery, calculates room heater sizing for fireplaces suggestions, views braai and fireplace options and submits general or product specific enquiries.

Staff

A Inflame employee who logs into the dashboard, manages incoming enquiries and custom requests, views the products details, updates enquiry status and updates product listings.

Admin

A superset of Staff with additional permissions such as creating or removing staff accounts, deleting web enquiry records, uploading project photos and adding, editing or deleting catalogue products with full price control.

User Stories

Customer

- As a Customer, I want to browse separate product categories for Fireplaces and braais, so that I can easily find the specific heating fireplace of braai that I'm interested in before asking for an enquiry or quote.
- As a Customer, I want to view a photo gallery of previous installations, so that I can assess the quality of Inflame products before committing to a purchase.
- As a Customer, I want to submit an enquiry at any time of the day, so that my enquiry is received as soon as possible.
- As a Customer, I want to submit a specific enquiry connected directly from the specific product page, so that the Inflame team automatically knows which specific product I am interested in.
- As a Customer, I want to enter my room dimensions and receive a recommended heater size, so that I know which fireplace will be the best option for my space before committing to a quote.
- As a Customer, I want to see the options available for my custom braai, so I am aware of the options before submitting an enquiry form.

Staff

- As a Staff member, I want to log in securely to a staff dashboard, so that only authorized employees can manage web lead data.
- As a Staff member, I want to view incoming enquires and custom enquiry requests, so that I can respond and organize further contact.
- As a Staff member, I want to view the attached product context in an enquiry, so that I can provide an accurate response based on the exact fireplace or braai selected.

- As a staff member, I want to update the status of an enquiry, so that the team can track follow-ups and avoid duplicate handling.

Admin

- As an Admin, I want to create, manage and remove staff accounts, so that access to the system remains secure and up to date
- As an Admin, I want to delete sensitive or outdated customer records, so that the system maintains data compliance and cleanliness.
- As an Admin member, I want to add, edit, and delete products in the catalogue, so that the website always reflects current stock and pricing.
- As an Admin member, I want to upload high resolution photos of completed projects to the online gallery, so that potential clients can see recent work.
- As an Admin member, I want to view and update customer records, so that I can track purchase history and follow up on servicing opportunities.
- As an Admin member, I want to view incoming enquiries, so that I can respond and organize further contact.

User Experience Journey Map

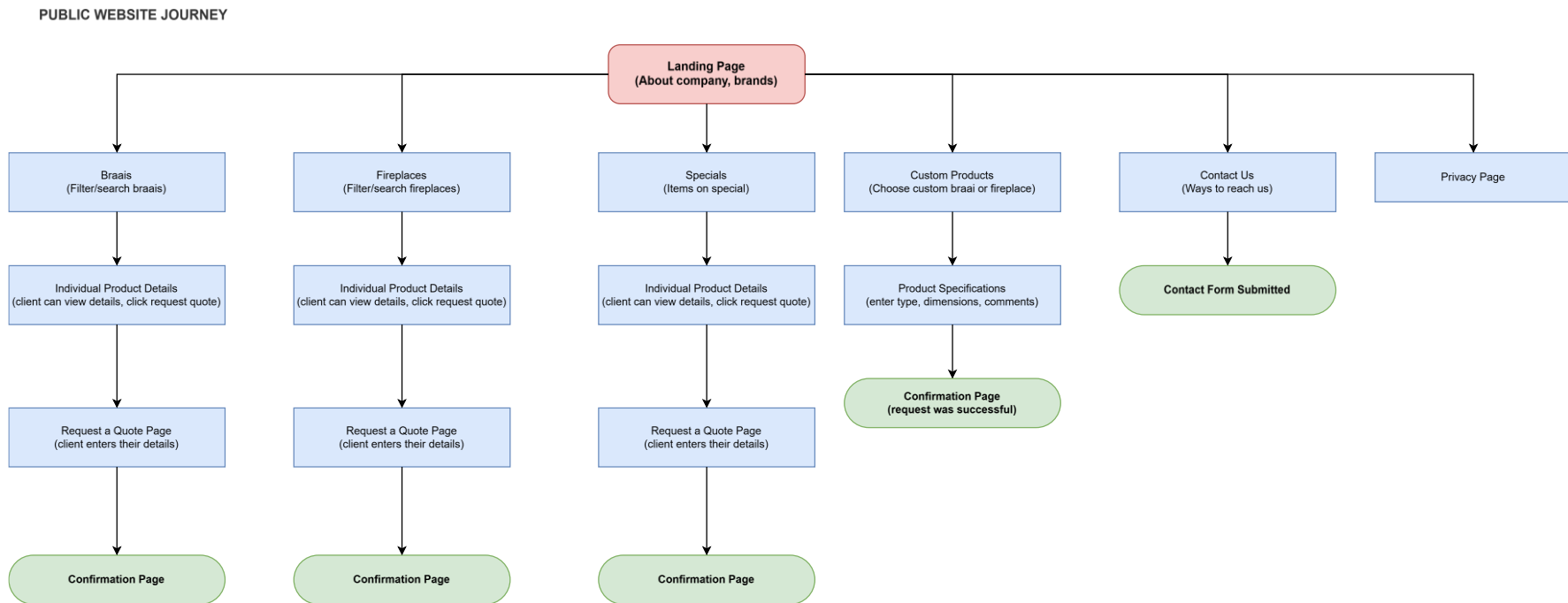


FIGURE 1: PUBLIC WEBSITE UX JOURNEY MAP

Admin UX journey map

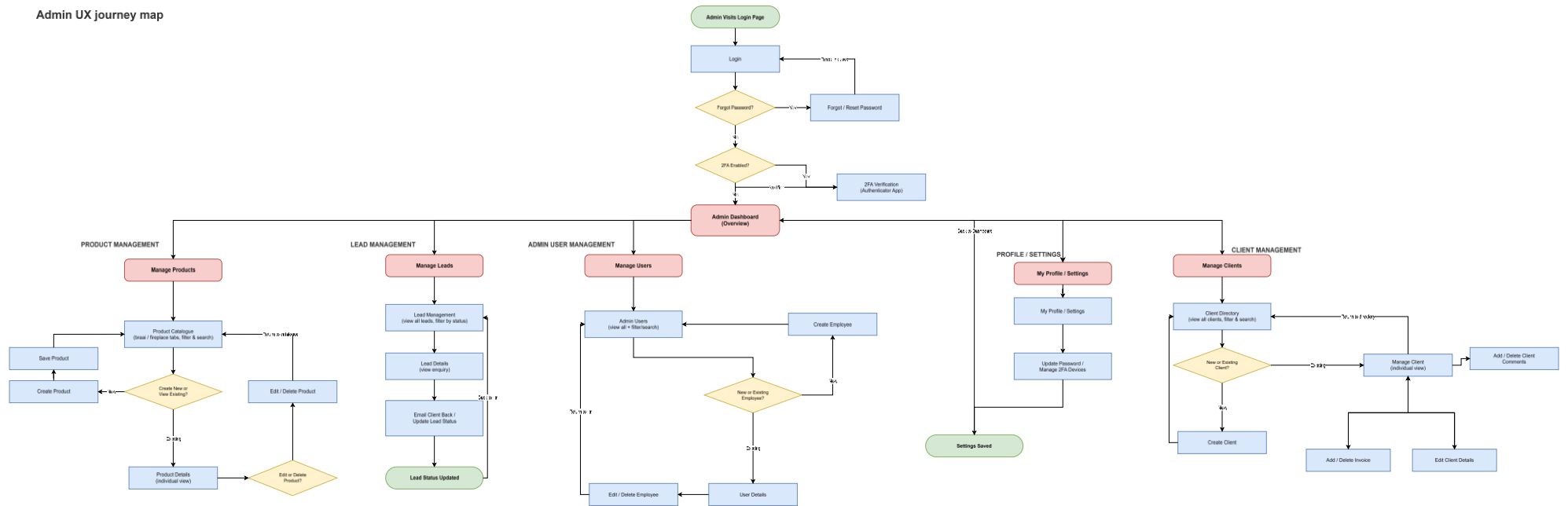


FIGURE 2: ADMIN UX JOURNEY MAP

Work Breakdown Structure

Task Name	Description	Start date	End date	Duration	Predecessor	Assignee	Resources needed
Introduction (KAN-6)	Define the project scope, background, and initial overview documentation.	24/07/2026	25/07/2026	1 day	None	Thomas Newman	Project Brief
User roles (KAN-8)	Describe the various user roles that your team has identified.	24/07/2026	26/07/2026	2 days	None	Leo Addinall	Stakeholder interview notes
Requirements (KAN-7)	Include all requirement artifacts in this section. Include any research that your team did to better understand the organisation, project and users of the software you will build.	26/07/2026	28/07/2026	2 days	KAN-6, KAN-8	Connor McCure	Jira artifacts, research notes
Non-functional requirements (KAN-12)	This section should cover non-functional requirements gathered through interaction with your client.	28/07/2026	30/07/2026	2 days	KAN-7	Connor McCure	Client meeting notes

User stories (KAN-9)	UX journey map A User Experience (UX) Journey Map shows all the screens in your software application and how a user can navigate between these screens. Consider doing this for each application that you plan to build and remember to include different user roles. • Work breakdown structure Draw up a table containing: - o The names of the tasks - o Description of each task - o Duration of each task - o Predecessor(s) if each task - o Team members responsible for each task - o Resources needed for each task	28/07/2026	30/08/2027	3 days	KAN-7	Leo Addinall	Draw.io
Data Schema/ER D (KAN-21)	Document the data storage explaining your choices of technology and develop an Entity	31/07/2026	02/08/2026	2 days	KAN-9, KAN-12	Leo Addinall	Draw.io , database docs

	Relationship Diagram (ERD). Include JSON schemas if using a document store technology (NoSQL). Describe and explain the diagrams and schema's that you include.						
Analysis Artifacts (KAN-17)	- Domain modelling Develop a UML domain class diagram that represents the main concepts of the system. The diagram should include the core domain classes, their attributes, and relationships. Briefly describe the purpose of each class. - Design artifacts Develop a UML design class diagram based on the system requirements. The diagram must illustrate the implementation classes, including their attributes,	31/07/2026	03/08/2026	3 days	KAN-9, KAN-12	Nafees Sampson & Mathew Bowes	Draw.io

	methods, access specifiers, keys, and relationships.						
Architecture Artifacts (KAN-22)	Design Patterns (UML Object Diagram) Document your use of best-practice design patterns and explain why they were selected. This section should demonstrate how the patterns are applied within the system at an object or interaction level. Can use: UML Object Diagram (showing instances involved in the design pattern). Architecture Patterns (System Architecture Diagram) Document your choice of architecture pattern and explain why it was selected for the system. This section should	03/08/2027	05/08/2026	2 days	KAN-21, KAN-17	Mathew Bowes & Leo Addinall	Provider docs, draw.io

	present the overall structure of the application and how its major components are organised. Can use: System Architecture Diagram. • Cloud Architecture (Cloud Architecture Diagram) Describe your cloud architecture and discuss the technology decisions made. Diagrams should include the cloud services used and clearly show network segregation as well as the protocols used to communicate between services. You can use: Cloud Architecture Diagram.						
GitHub Actions Pipeline (KAN-28)	Describe, with the use of a diagram, the functionality of your GitHub Actions pipeline.	05/08/2026	07/08/2026	2 days	KAN-22	Mathew Bowes	Github repo, Github

	The description should clearly illustrate the branching model and workflow, how code changes move through the pipeline, and where automated testing is executed as part of the continuous integration and deployment process.						actions, Draw.io
Implementation (KAN-20)	This section should show your implementation documentation, such as UML sequence diagrams and UML state diagrams showing critical flows; deployment plan documentation; as well as any related documentation. Only important flows need to be documented.	05/08/2026	08/08/2026	3 days	KAN-22	Nafees Sampson	Github repo, draw.io
DevOps (KAN-27)	Task 1 is intended as an initial project plan for this section and is subject to change during later stages	07/08/2026	09/08/2026	2 days	KAN-28	Mathew Bowes	Github CI/CD pipeline

	of the project. Rapid deployment of a high-quality application is pivotal to the success of a software product. This section should discuss how your team has addressed this non-functional requirement in your project, with particular focus on your use of GitHub for version control, branching strategies and development flow, and the integration of automated testing to ensure quality and reliability throughout the development lifecycle.						
Security (KAN-26)	In this section, provide a detailed discussion of how security was addressed within the application. Your discussion must cover standard security considerations, including network security, common	09/08/2026	11/08/2026	2 days	KAN-27	Mathew Bowes	Security testing tools

	vulnerabilities, and potential threats. Specific attention should be given to how the application prevents and mitigates cross-site scripting (XSS) attacks, along with any other relevant security risks.						
Change Management (KAN-30)	This section should describe your approach to managing the adoption of the software product your team has built. Consider addressing the following questions: • How and why will the organisation adopt your software? • How and why will the users adopt your software? • What is your strategy to gain adoption from	08/08/2026	11/08/2026	3 days	KAN-20	Thomas Newman	Stakeholder communication plan

	both the organisation and the users? In addition, include a plan for handing over the system to the client, detailing the transition from development to production. Describe how users and the organisation will be moved from their old system to the new system, including training plans, timelines, and support strategies to ensure a smooth adoption process.						
Running costs (KAN-29)	Show the predicted monthly costs of running the solution. Include full running costs, such as hosting, databases (e.g. SQL, Firebase), and any other infrastructure services required to operate the application. The solution must be	09/08/2026	11/08/2026	2 days	KAN-27	Leo Addinall	Provider pricing research

	hosted, but do not include costs related to team fees or development effort. Show growth projections for two (2) years, based on input from the client. Students are also required to provide a table illustrating costs at different scales, for example: 100 users, 1,000 users, and 10,000 users, considering typical free-tier allowances						
Prototype (KAN-31)	The prototype must demonstrate the following: • Design: Show the visual layout and interface of the system, including navigation elements and key screens. • Flow of Information: Illustrate how data moves through	11/08/2026	14/08/2026	3 days	KAN-20, KAN-26	Whole Team	Figma

	the system using dummy data where appropriate, so that functionality and user interactions can be clearly understood. • Responsive Design: Include both mobile and desktop versions of the interface to demonstrate how the system adapts to different devices. The goal of this prototype is to provide a clear, functional representation of the system for both stakeholders and developers, highlighting design decisions and user experience.						
Final review and hand in	Hand in task 1	15/08/2026	15/08/2026	1	None	Whole team	

Non-Functional Requirements

Performance

Both the public website as well as the room size calculator should return results to the client in under two seconds. The API responses for the owner dashboard when fetching order histories should also have a latency of less than 200ms with a response time of under 300ms.

Due to Inflamm currently losing out to competitors, a website that is fast loading is critical to keep potential clients engaged with what Inflamm has to offer. For the owner, a dashboard that is highly responsive is essential for keeping up in the fast-paced working environment of manufacturing these braais and fireplaces where fast turnover times are vital when aiming for customer satisfaction. Any solution that slows the owners working speed down, would be seen as a hinderance rather than a benefit to the company.

Scalability

The website solution will incorporate horizontal scaling to support Inflammes goal of having a 10x increase in traffic over the next two years. To achieve scalability for the owner dashboard, the solution must be able to handle a 10x increase in stored order information without exceeding the 300ms response time mentioned above.

With Inflamm aiming to have a constant influx of new and returning customers, the solution will need to grow with the business without needing to be completely redesigned as a system redesign is not feasible for a small family-owned business like Inflamm. The owner dashboard being scalable is essential to growing the business as if the dashboard cannot scale with an increasing history of orders, this will lead to the dashboard becoming a bottleneck leading to the owner being better off returning to his old ways of storing order history.

Availability

Both the website solution as well as the owner dashboard should maintain a 99.9% uptime and will aim to be fully operational during working hours 8:00 – 17:00 as well as paying particular attention to maintaining uptime during the pre-winter and winter months in Cape Town. Automated daily cloud database backups will also be completed which gives Inflamm a recovery point objective of less than or equal to 24 hours.

As discussed with the owner of Inflamm during our interviews, it is of vital importance that the website solution as well as the dashboard maintain uptime during business hours and especially during the pre-winter and winter months as he informed us that most of his work for the year comes during these months as the demand for fireplaces increases during the colder months. It is also vital to ensure backup mechanisms are in place while transitioning Inflamm to a digital environment for the storage of their order histories and paperwork to ensure important order information is not lost.

Reliability

To ensure reliability of the solutions, retry logic must be implemented in the contact form to ensure that potential leads are not lost if the client submits their details while the server is down and to ensure the user experience is not interrupted. In the owner dashboard, all POST operations should incorporate self-cancelling if for example the owner clicks save invoice twice due to slow connection speeds, the system should be reliable enough to make sure only one instance of that invoice is created and saved in the database. The solution should also be reliable enough to provide consistent and accurate results when clients are using the room size calculator regardless of the current website traffic.

It is vitally important that retry logic is implemented in the contact form to ensure that the details of every person interested in a braai or fireplace actually reaches the owner regardless of server availability to ensure the business does not miss out on potential sales. During POST operations in the owner dashboard, if the solution creates duplicate records or fails to create a record during a network error, this could frustrate the owner and waste important time corrected these mistakes. The room size

calculator being reliable enough to display accurate information regardless of traffic is vital as it allows the owner to not have to correct system mistakes during follow up calls, allowing him to focus on closing the sale.

Security

To ensure security of data, all the data transmitted for both the website solution and the owner dashboard must be encrypted using TLS 1.3 (HTTPS) as well as AES-256 for data at rest and role-based access control for access to the owner dashboard as well as session management. The client contacts forms and all inputs done must also include strict validation. The owner dashboard will also be protected with a dedicated username and encrypted password.

The owner dashboard will store sensitive information such as invoices, paperwork and client and order history so securely handling these documents is extremely important to Inflamm as well as to ensure Inflamm complies with POPIA and to protect data from unauthorized external access.

Maintainability

Both solutions will be developed in a modular and layered architecture manner to ensure one part of the system can be updated without breaking the rest. API endpoints will also be documented with clear instructions on how different parts of the system talk to each other. Clear service boundaries will be incorporated to ensure every part of the system has a dedicated job and performs that job effectively. Logging will be implemented for all solution errors and operations performed by the database to simplify the troubleshooting process. Automated CI/CD workflows through GitHub actions must also allow bug fixes as well as updates to the system to be done with minimal to zero system downtime.

Due to Inflamm being a small family-owned business, it does not have a dedicated IT department which makes maintainability even more important as the system will need to be built cleanly so maintenance in the future can be done without breaking the existing system and allowing different developers to easily pick up from where TCMNL solutions have left off. The automated GitHub actions also allows for automated code

testing to be done to prevent broken code from reaching the live solutions, ensuring the systems uptime.

Analysis Artifacts

Domain Modelling

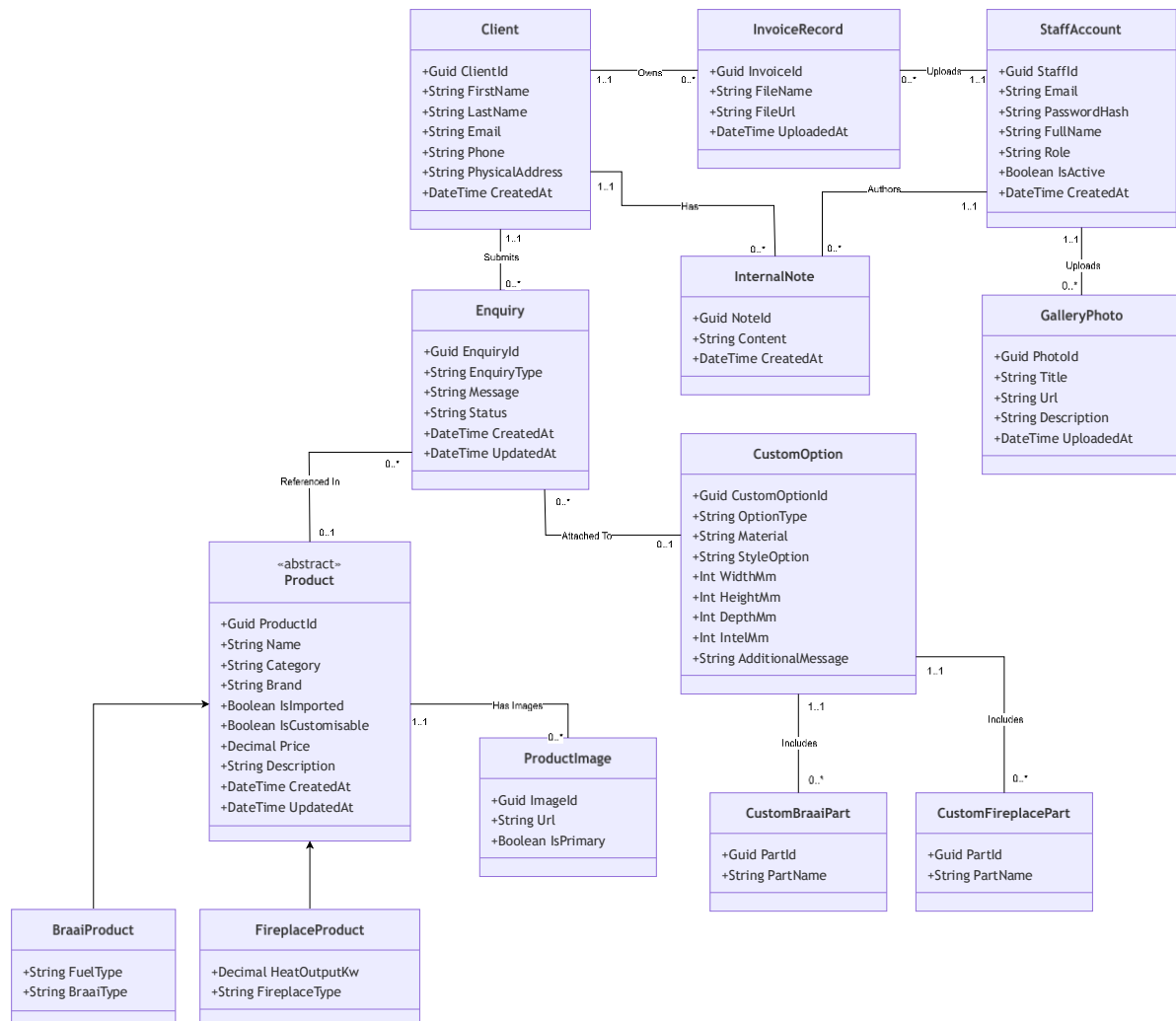


FIGURE 3: UML DOMAIN CLASS DIAGRAM

Description:

A Client enquires about a Product, which would either be a BraaiProduct or FireplaceProduct. Alternatively, a client can describe a CustomOption if they want something custom built instead. A client is always doing one or the other, never both so an Enquiry is either about a catalogue Product or a CustomOption (with its related CustomBraaiPart/CustomFireplacePart components). Each Enquiry captures what the

client wants and tracks its current status. On the business side, a StaffAccount reviews enquiry, can log InternalNotes about the client, and can upload an InvoiceRecord at any point in the process, not just once a final decision is made. Staff also manage the catalogue product photos (ProductImage).

Design Artifacts

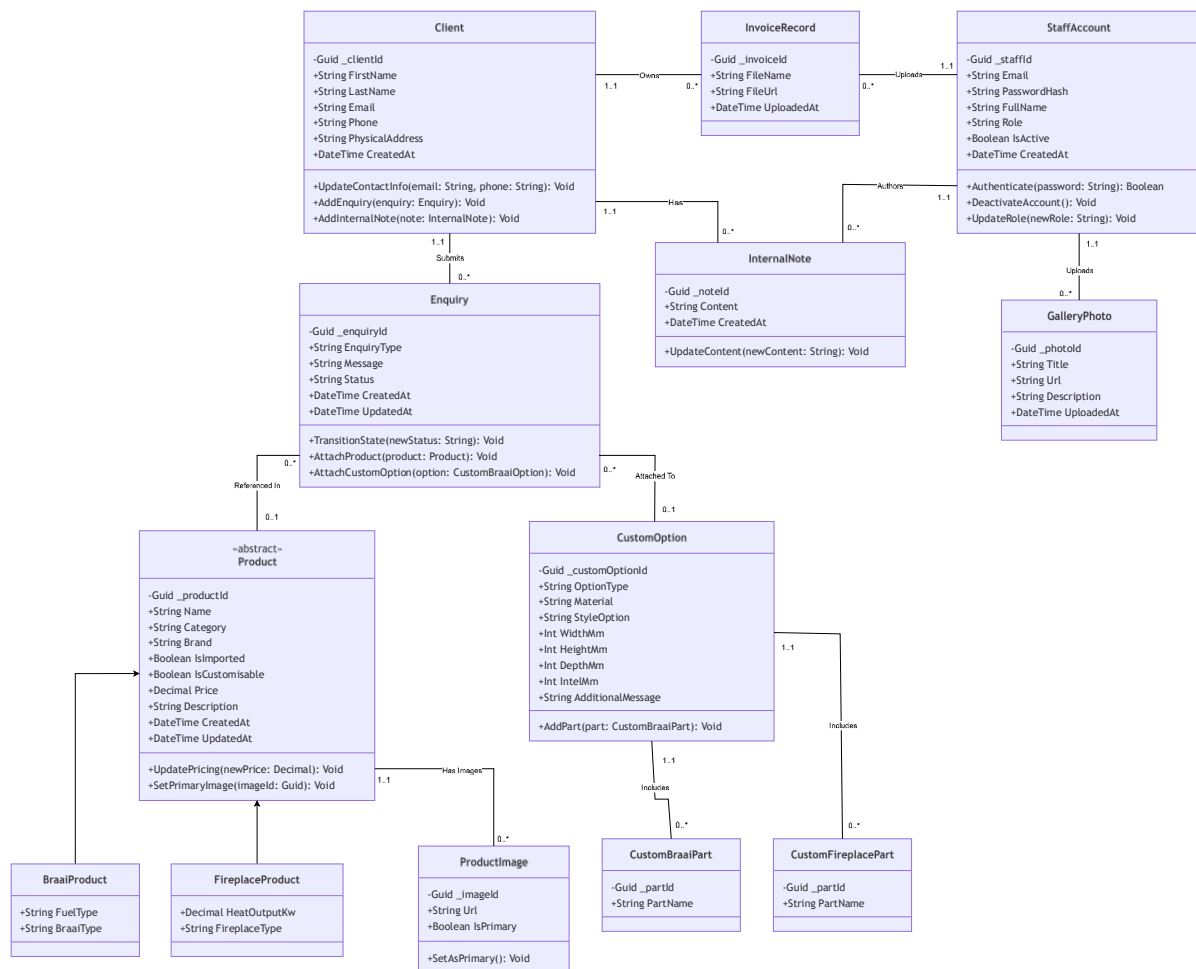


FIGURE 4: UML DESIGN CLASS DIAGRAM

Description:

The design follows the same structure as our domain model but adds more detail for implementation. Every entity has a Guid primary key, Client, StaffAccount, and Product include CreatedAt and UpdatedAt fields to keep track of when records are created and changed. Enquiry.TransitionState() handles status changes,

`AttachProduct()` and `AttachCustomOption()` ensure that an enquiry can only be linked to a catalogue product or a custom build. `StaffAccount.Authenticate()` handles the actual staff login, using `PasswordHash`, `IsActive`, and `Role` instead of the simpler staff model used previously. Uploaded files such as product images, gallery photos, and invoices are represented as URLs through `ProductImage.Url`, `GalleryPhoto.Url`, and `InvoiceRecord.FileUrl`. This means the actual files are stored using an external storage service rather than directly in the database.

Implementation

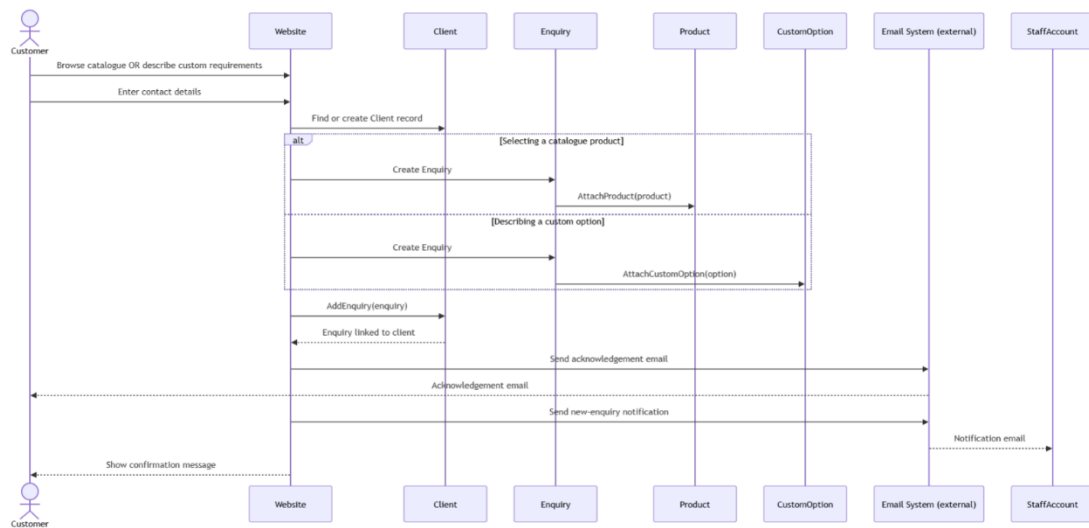


FIGURE 5: UML SEQUENCE DIAGRAM : CLIENT SUBMITS AN ENQUIRY

Description:

This sequence diagram shows the flow of a client submitting an enquiry through the Inflamm website. The customer either selects a product from the catalogue or requests a custom build, then enters their contact details. The system finds their existing Client record or creates a new one, and depending on their choice, the Enquiry is linked to a Product via `AttachProduct()`, or to a CustomOption via `AttachCustomOption()` but never both at once, since these two paths are mutually exclusive. Once the enquiry is linked to the client via `AddEnquiry()`, the system sends an acknowledgement email to the customer and a notification email to staff before confirming the submission on screen.

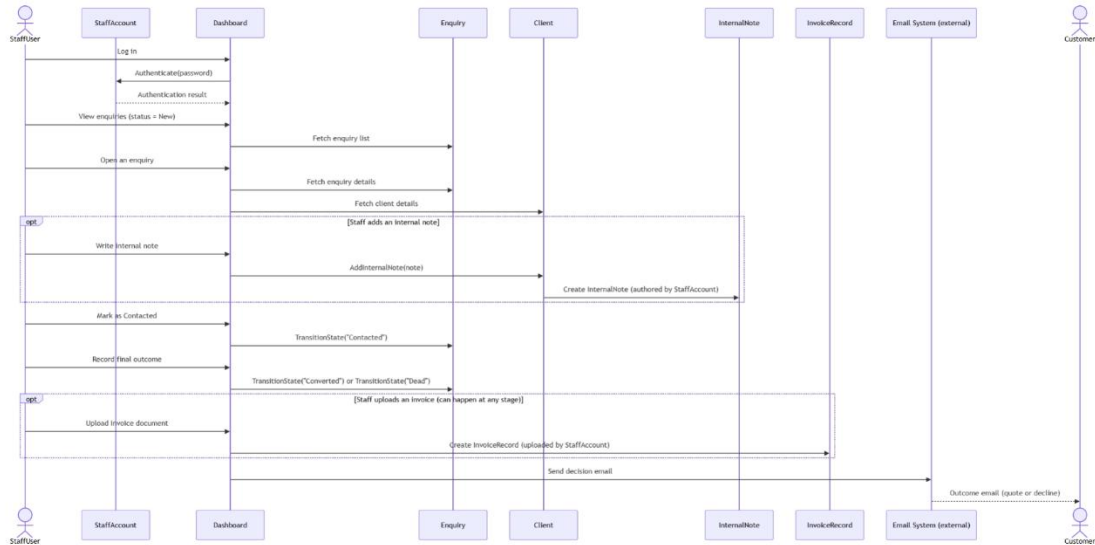


FIGURE 6: UML SEQUENCE DIAGRAM: STAFF REVIEWS AND DECIDES ON AN ENQUIRY

Description

This sequence diagram shows how a staff member processes an enquiry from the dashboard. After logging in and being authenticated, staff can view the list of new enquiries and open one to see the client and enquiry details, optionally adding an internal note via `AddInternalNote()`. The enquiry is transitioned to "Contacted" once staff reaches out to the client and after an off-system consultation or site visit, staff records the outcome by transitioning the enquiry to either "Converted" or "Dead" using `TransitionState()`. An invoice can be uploaded at any point in this process, creating an `InvoiceRecord`. At the end of this process a decision email sent back to the customer either being a quote or rejection.

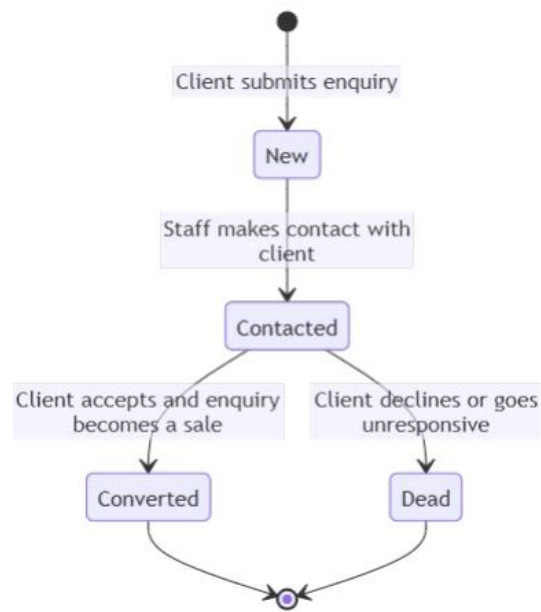


FIGURE 7: UML STATE DIAGRAM: ENQUIRY STATUS LIFECYCLE

This state diagram represents the possible statuses an Enquiry can move through over its lifetime. Every enquiry begins as New the moment it's submitted. Once a staff member reaches out to the client, it moves to Contacted. From there the enquiry ends in one of two final states either Converted, if the client accepts and it becomes a sale, or Dead, if the client declines or stops responding. Both Converted and Dead are terminal states once reached; the enquiry's lifecycle is complete.

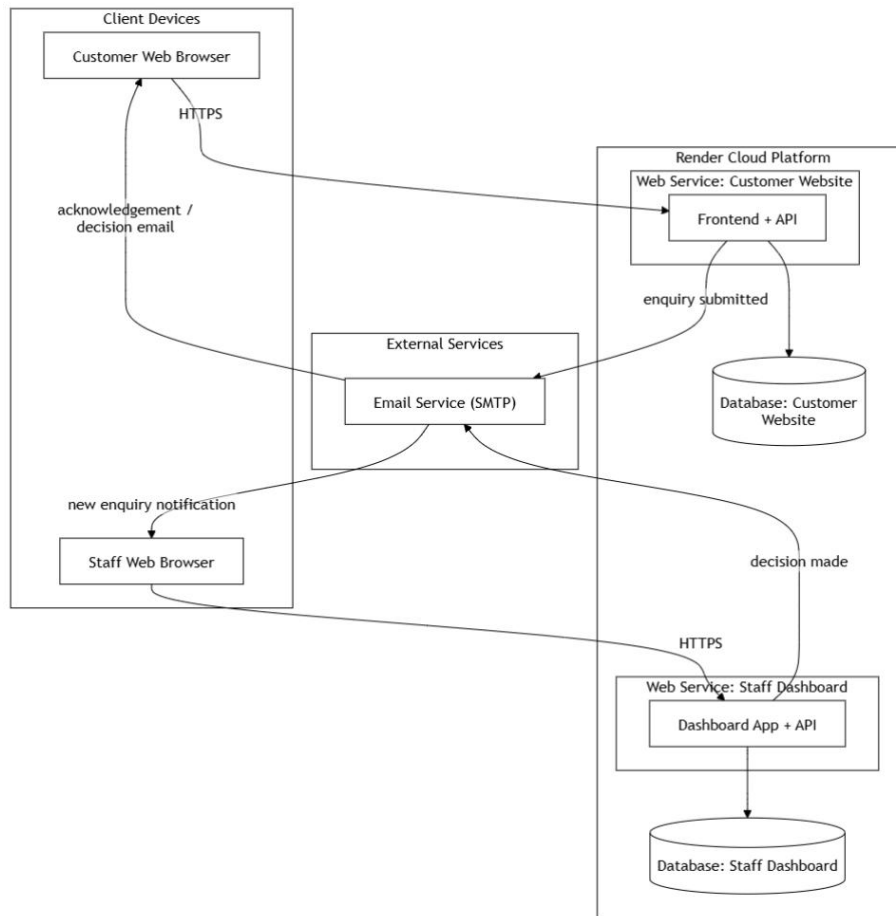


FIGURE 8: DEPLOYMENT DIAGRAM: SYSTEM HOSTING ON RENDER

This deployment diagram shows how Inflames system is hosted on Render, a cloud platform-as-a-service. The system is split into two independently deployed web services, matching the two-website structure of the project which are the Customer Website, where clients browse products and submit enquiries, and the Staff Dashboard, where staff log in to review and manage those enquiries. Each service connects to its own dedicated database, keeping the public facing site and the internal dashboard isolated from one another. Both the customer's browser and the staff member's browser connect to their respective service over HTTPS. An external email system handles all outgoing communication acknowledgement emails to customers, new-enquiry notifications to staff, and final decision emails rather than either web service sending mail directly itself.

Deployment Plan

Platform

We decided to deploy Render, which is a Platform-as-a-Service provider. The main reason for this choice is that it is easier to set up and maintain our own servers. It also comes with HTTPS built in and connects directly to GitHub, so any push to the main branch automatically redeploys the app. Since we're a small student team without anyone dedicated to managing infrastructure, Render's free tier made this an easy and practical choice for us.

Environment Setup

The system is split into two independently deployed Render Web Services, reflecting the two-website structure of the project:

Customer Website Service - hosts the public-facing site where clients browse products and submit enquiries. Connected to its own dedicated database instance.

Staff Dashboard Service - hosts the internal dashboard used by staff to authenticate, review enquiries, update statuses, and upload invoices. Connected to its own separate database instance.

Each service has its own set of environment variables, the database connection string, SMTP credentials, and authentication secrets keep the two systems properly isolated from each other. We're also not handling email sending directly from either app server. Instead, an external SMTP provider takes care of all outgoing email, whether that's the acknowledgement sent to a customer, the notification that goes to staff, or the final decision email once an enquiry has been reviewed.

Rollout Strategy

1. **Database provisioning** - create both database instances on Render and run initial schema migrations for each service
2. **Backend/API deployment** - deploy the Customer Website and Staff Dashboard services separately, each connected to its own database
3. **Environment configuration** - set environment variables (DB connection strings, SMTP credentials) per service via Render's dashboard, keeping secrets out of source control

4. **Testing** - verify the enquiry submission flow end-to-end (client submits - email sent - appears on staff dashboard) in a staging/preview environment before promoting to production
5. **Go live** - point production domains at both Render services and monitor initial traffic and email delivery
6. **Ongoing deploys** - future changes are deployed automatically via Git push to the main branch, with Render rebuilding and redeploying each service independently

Rationale

We split the system into two separate services with their own databases because it reflects how these two parts of the system function in the real world the staff dashboard should never be publicly accessible, and if one service goes down, it shouldn't take the other one with it. Choosing a managed PaaS over self-hosted infrastructure was a deliberate. We gave up some of the low-level control you'd get from managing your own servers, but in return we got a much faster and simpler deployment process, which made sense given our timeline and the fact that none of us have much experience running server infrastructure.

Data Schema

The Inflamm website needs to store a product catalogue of braais and fireplaces, customer submitted enquiries, staff and admin login accounts and a photo gallery of completed installation. The data is interlinked with an enquiry mostly always needing to point back to the specific product or custom configuration and a products category determines which extra attributes it carries.

Primary data storage:

PostgreSQL accessed through the managed Supabase platform was chosen as the primary data storage for the following reasons:

- Enforces referential integrity natively, which is important because most enquiries, products and custom configurations all depend on each other.
- Provides Strong Transactional guarantees meaning an admin editing a product or updating an enquiry status can't leave the database in a half-written state.
- Support in the .NET C# ecosystem via Entity framework Core which makes the data layer straightforward to build and query.
- Supports Enums, JSONB columns and row level security which gives flexibility for semi structured fields without leaving the relational model.
- Supabase is chosen as the managed layer because it bundles hosted PostgreSQL, authentication and object storage behind one service and one C# client library which reduces how many separate providers the team has to set up and maintain

Caching: Redis via Upstash

Redis was chosen as the Caching layer because the public catalogue and gallery will be read far more often than written meaning an in-memory cache in front of PostgreSQL keeps those pages fast without adding load to the primary database. Redis also specifically offers key expiry and simple data structures that will suit cached JSON blobs of catalogue/gallery data.

File/Object Storage: Supabase Storage

Supabase Storage was chosen as the File/object storage because product photos and gallery images are binary assets that don't belong in relational rows meaning they are stored in a bucket and referenced by URL from the database.

Scalability

PostgreSQL can scale vertically first then horizontally via read replicas for public catalogue traffic plus connection pooling for concurrent users. Redis can move to a clustered/relicated setup if read volume grows without changing how the application talks to it. Object Storage scales independently of the database which is important since photo volume will likely grow faster than transactional data.

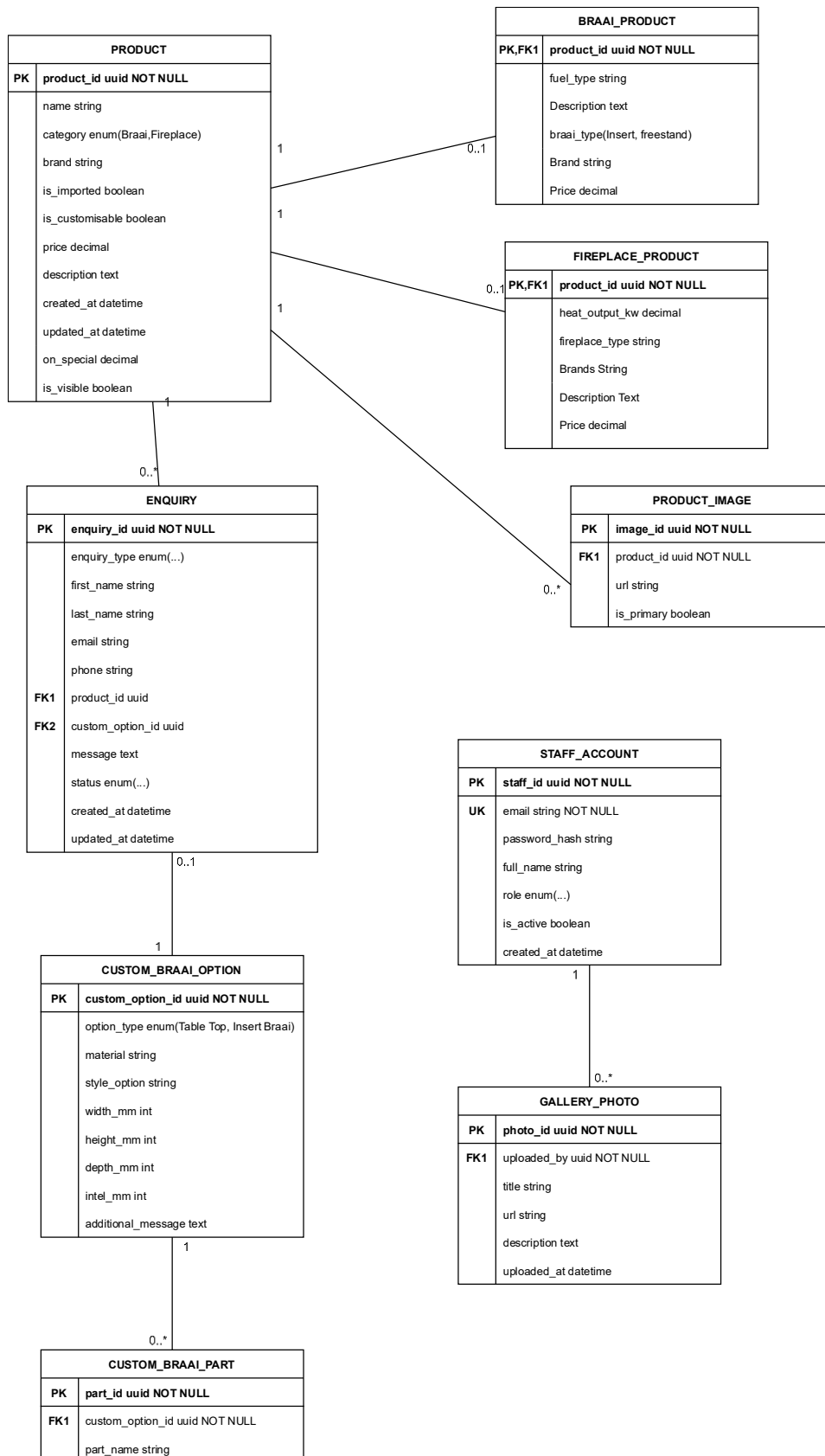


FIGURE 9: ENTITY RELATIONSHIP DIAGRAM

Architecture Artifacts

Design Patterns

Singleton Pattern (Creational)

The singleton design pattern has been selected as it allows for a single instance of a class to be created and used throughout the life of the application [1]. This design pattern is to be used within the C# .NET Core Web API application which will allow for a single shared instance of the Redis connection multiplexer and the Supabase client [2, 3]. Each instance will be registered as a singleton in the Dependency Injection (DI) container where the lifecycle of the object will be managed [4]. Using a singleton for both the Caching and Database connections ensures that all calls are made through the same instance which allows for resource optimization and prevents the possibility of redundant database connections [5].

Factory Method Pattern (Creational)

The factory method pattern provides an interface for creating objects, which allows subclasses to alter objects type before being created [6]. This pattern will be used for product creation. The application has a base product which can branch into different types such as Braai and Fireplace, a factory will handle the instantiation of the products ensuring that the code is clean and easy to maintain [6]. The factory will handle all the necessary validation and mapping ensuring the correct product is created.

Builder Pattern (Creational)

The builder pattern allows complex objects to be constructed step by step, which allows for an object of a different type or representation to be produced using the same construction code [7]. A variation of this pattern called “Query builder” will be used to implement dynamic product search and filtering [8]. If various filters are applied such as price range, material and brand, the Entity Framework query will become messy. Using the builder pattern queries can be created dynamically using LINQ [9].

Adapter Pattern (Structural)

The adapter pattern allows objects with incompatible interfaces the ability to interact with each other [10]. The adapter pattern will be used to integrate the third-party cloud storage services. This pattern will allow our product class and Superbase endpoint to be mutually unaware of each other. This decouples the service usage, which will allow for service providers to be interchanged in the event of growth.

Decorator Pattern (Structural)

The decorator pattern allows for new behaviours to be attached to objects by placing the object inside a special wrapper object which contains the new behaviour [11]. Our application will be using Redis to cache the product catalogue to ensure quick loading, implementing this pattern will ensure the database manager does not contain any caching logic. The decorator pattern will implement the database manager interface where the cache adapter and base class is injected. This ensures that the cache is checked first bypassing the database and falling back to the database when the required data is not found.

Facade Pattern (Structural)

The façade pattern provides simplified interfaces to libraries, frameworks or other complex sets of classes [12]. This pattern will be applied to the services within the application such as a ProductCatalogueFacade. This implementation ensures that the API controllers are kept clean, and without any business logic. The facade acts as an additional layer between the controller and service layer helping to keep the codebase clean and scalable [12].

Command Pattern (Behavioural)

The command pattern transforms requests into standalone objects which contain all the information about the requests that were made [13]. This allows requests to be passed as a method argument, delayed or queued the execution of the request [13].

Our application makes use of an admin portal (which writes data) and a public website (heavily reads data) [14]. Implementing this pattern will ensure that the controllers are decoupled from the business logic. The Command Query Responsibility Separation (CQRS) pattern will be utilized for the write-side of the application, which will help with security by separating the read and writes [14].

Mediator Pattern (Behavioural)

The mediator pattern reduces chaotic dependencies between objects, which restricts any direct communication between the object forcing interactions to be done through the mediator object [15]. This pattern will help to implement the CQRS pattern ensuring that both command and queries are mapped to their respective handler [16]. This pattern will force the API controllers to go through the Mediator which helps to decouple the controllers. Although the use of the mediator pattern help with decoupling it can evolve into a god object [15].

Observer Pattern (Behavioural)

The observer pattern defines a subscription mechanism where multiple objects can be notified about any events that transpire to the object being observed [17]. This pattern will be implemented in the “Get Quote” feature in the individual product view. When this feature is used, the lead should be saved to the PostgreSQL database, where it is logged for analytics as well as an email to a member of staff to follow up on the lead.

State Pattern (Behavioural)

The state pattern lets objects alter their behaviour when its internal state changes, which appears as if an object is simply changing its class [18]. This pattern will be implemented for the lifecycle of a QuoteLead. Where the quote enters the system and is then managed on the admin portal. A state interface will be used to define the actions that can be used on a lead. Implementing this pattern ensures that a lead cannot move backwards in terms of its state.

Chain of Responsibility Pattern (Behavioural)

The chain of responsibility pattern allows requests to be passed along a chain of handlers, where each handler decides whether to process or pass the request along the chain [19]. .NET Core already implements this pattern through the built-in middleware [20]. The middleware allows the application to implement authentication and rate limiting.

UML Diagram

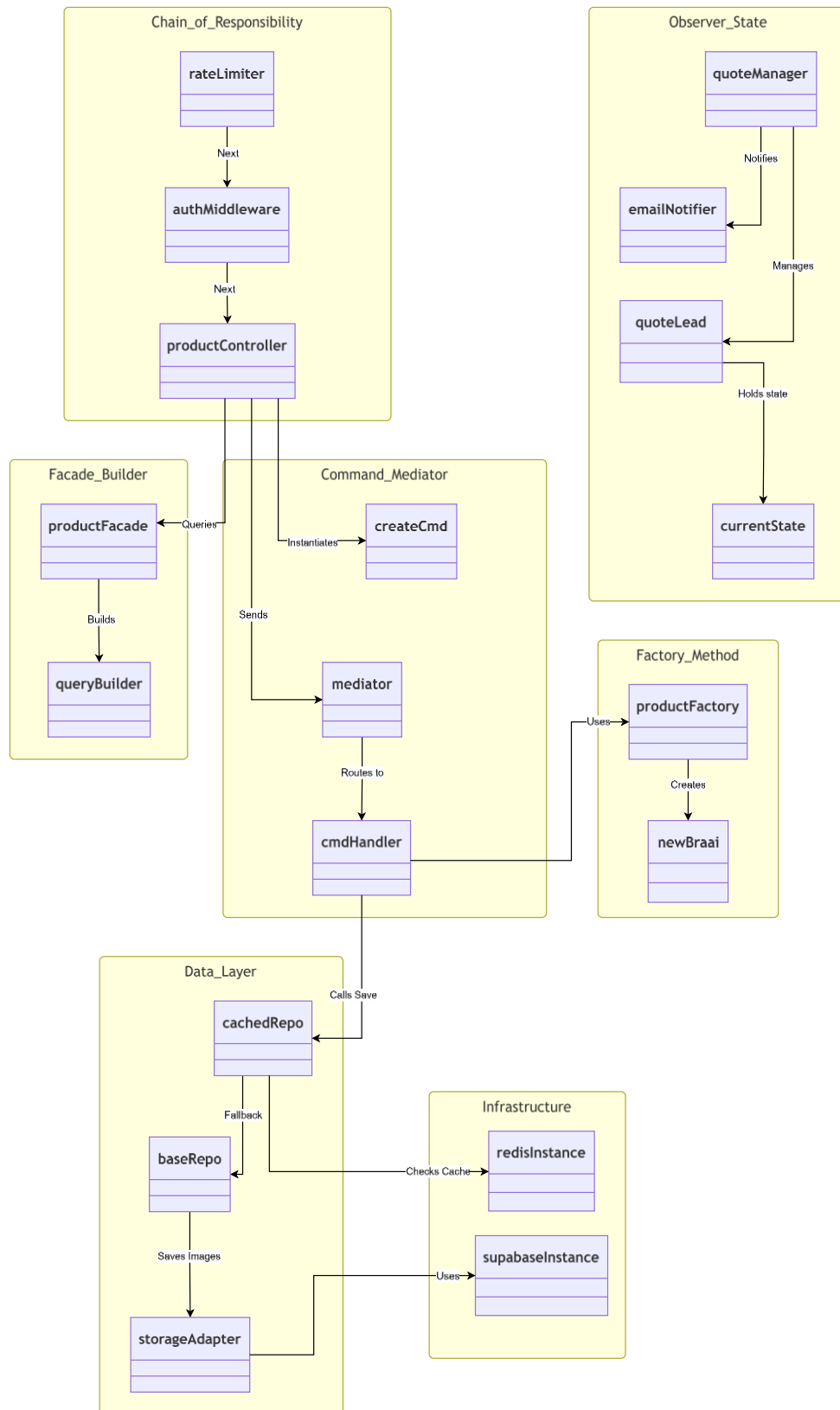


Figure 10: UML Diagram for Design Patterns

Architecture Patterns

Layered (N-Tier) Pattern

The Layered architecture pattern organizes an application's components into horizontal layers, which includes the presentation, business and data access layers [21]. The application we are creating will utilize this pattern allowing our controllers to be kept clean where all business logic can be pushed to the Mediators and Facades, which then relies on the data access layer. This pattern ensures that both features are easier to implement and isolate later in development [22].

CQRS (Command Query Responsibility Segregation) Pattern

The Command Query Responsibility Segregation (CQRS) pattern separates read and write operations into separate models which can be optimized independently [21]. Utilizing this pattern separates the read and write workloads of our application. The Command and Mediator design patterns handle the write-side of the application which is for the admin portal, and the Facade and Builder patterns are used for the read-side of the public website. This pattern also ensures efficiency allowing the system to handle more work while avoiding any issues [22].

Repository Pattern

The Repository pattern creates a middle layer which handles the data access which ensures that managing and changing data sources is easier [22]. The data access layer of our application relies completely on this pattern where the decorator pattern (cachedRepo) and base class are used as the abstraction layer which ensures that the business logic is unaware of any queries. This pattern ensures efficiency and reliability for the database [22].

Client-Server Pattern

The Client-Server pattern is an architecture which comprises of the client and server [23]. The client requests a service from the server, and the server provides the service

[23]. Our application will make use of the API-first approach which follows this pattern [24]. The frontend and admin portal are the clients who are requesting the data, while the C# .NET Core Web API is the server which executes the business logic. This pattern ensure the clients side is lightweight, while the server handles the intensive tasks [22].

System Architecture Diagram

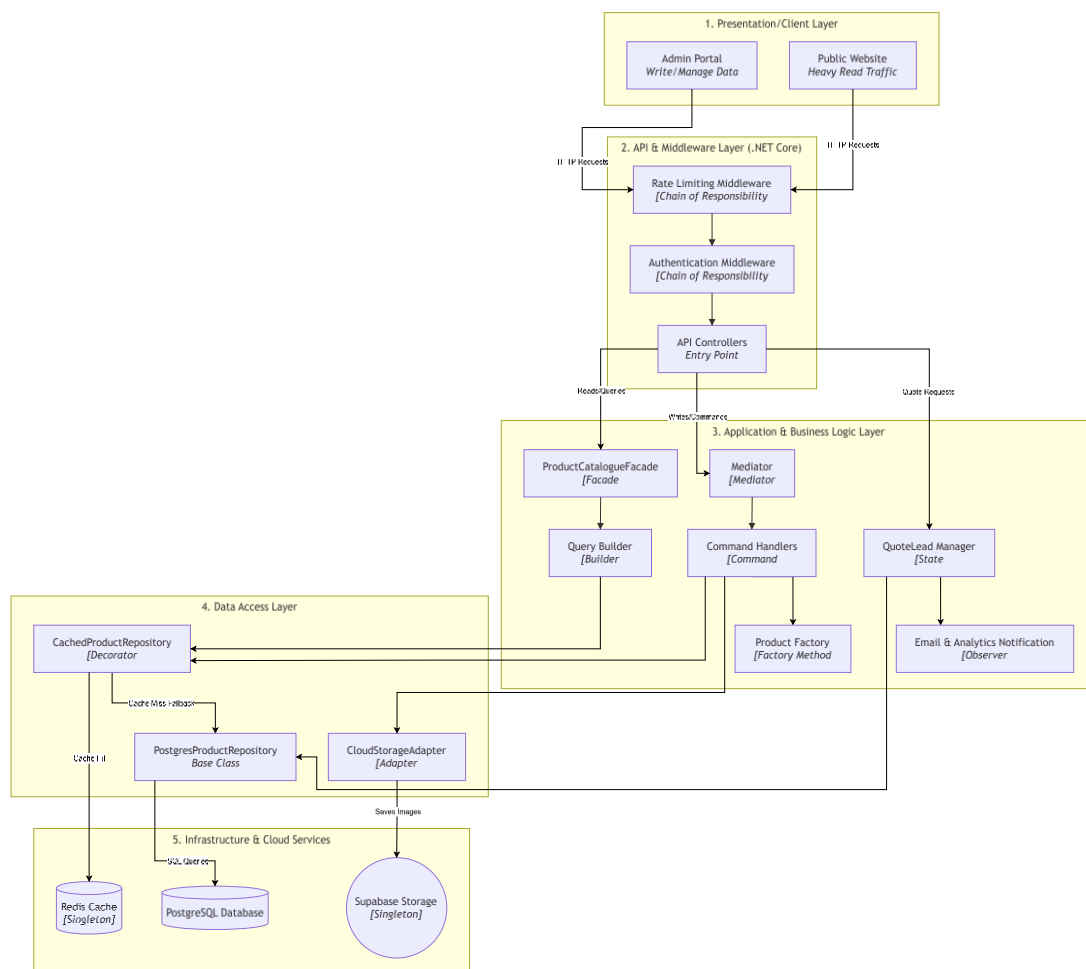


Figure 11: System Architecture Diagram

Cloud

The application will be deployed using a fully managed service composed of cloud architecture with each layer of stack such as compute, caching, database and file storage being delegated to a separate managed platform with the API acting as the single point of contact between clients and every backing service [25]

The C# .NET Core Web API is packaged as a Docker container and hosted on Render. Containerising the API means the application with its runtime and its dependencies are bundled into one portable image that runs identically regardless of the underlying [26]. Render builds and runs this container directly from the projects Docker file, redeploying automatically on every push and exposes only a single configured port to the public internet with every other interaction the API has with its backing services is initiated outbound, never inbound [27]

Caching is provided by Upstash which is a serverless Redis platform. It bills per request rather than for a continuously running instance and supports the standard Redis Protocol over tls so the APIs existing Redis client configuration works against it unchanged [28]

The database and file storage are provided by Supabase comprising a managed PostgreSQL instance and a S3 compatible object storage bucket for product images [25]

Cloudflare sits in front of the application as a content delivery network and Web application firewall, running on its free tier. This still includes DDoS protection, global edge caching and a baseline managed WAF ruleset at no cost [29]

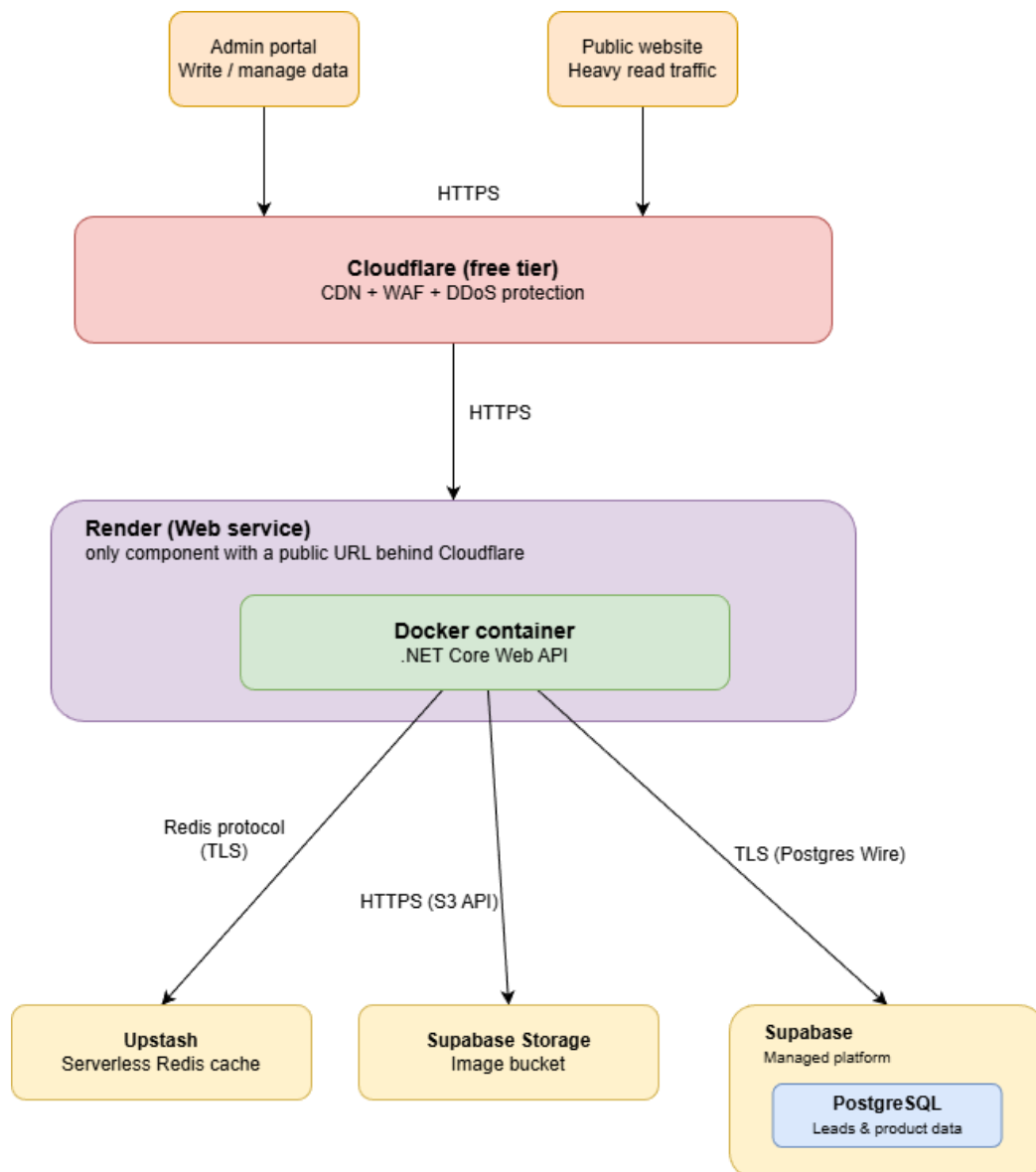


FIGURE 12: CLOUD ARCHITECTURE DIAGRAM

Network Segregation

Segregation here is enforced at the service and credential level rather than through network subnetting. Render is the only component ever reachable from a client request via Cloudflare. Upstash and Supabase are never contacted directly by the Admin Portal or Public Website only by the APL every connection from the API to a backing service requires both an authentication credential and TLS encryption so a service being reachable over the public internet does not mean it is reachable without authorization.

Protocols between Services

Client to Cloudflare: HTTPS

Cloudflare to Render: HTTPS

API (Render) to Upstash: Redis protocol over TLS [30]

API (Render) to Supabase PostgreSQL: PostgreSQL wire protocol over TLS [31]

API (Render) to Supabase Storage: HTTPS (S3-compatible REST API)

Security

Cross-Site Scripting (XSS) Attacks

Risk

Cross-site scripting (XSS) is a security vulnerability which allows attackers the ability to compromise users' interactions with a vulnerable application [32]. XSS attacks allow attackers to masquerade as a victim user, which allows them to perform action that the user is able to perform with the addition of being able to access the users' data [32]. XSS attacks work by injecting malicious scripts into a website, which executes on the victim's browser which compromises their interaction with the application [32]. This form of attack is a risk to both our public website and admin portal, where a users' session can be stolen or product listing can be defaced.

Mitigation Strategy

XSS attacks can be prevented by implementing a strict Content Security Policy (CSP) header which restricts the resources (scripts and images) which a page can load [32, 33]. This can additionally be prevented by validating and sanitizing all user inputs, which ensures that all malicious scripts are cleaned before the page is rendered [34]. The validation should be done on the server-side as it cannot be manipulated like on the client-side [34]. Implementing these strategies in our application will help to ensure that XSS attacks cannot take place.

Unauthorized Access and Privileged Escalation

Risk

Unauthorized access and privileged escalation falls under access control, which is the application of constraints for what a person is authorized to do within an application [35]. This vulnerability is fully dependant on how the application manages authentication and sessions [35]. The risk of this vulnerability to our application, would be where an attacker gains access to the admin portal and has full product CRUD functionality, and the ability to view sensitive client lead data.

Mitigation

This vulnerability can be prevented by always denying access by default unless the resource is intended for public access [35]. Additionally, in our codebase we must ensure that for each resource, access is denied by default ensuring minimum access is given by default [35]. The backend of our application will enforce .NET Identity (default password hashing) with Two-Factor Authentication (2FA) and a strict Role-Based Access Control (RBAC). Implementing .NET Identity will ensure the admin accounts are secure with password hashing and 2FA. These strategies will ensure that admin accounts are secure and given the minimum access needed using RBAC.

Distributed Denial of Service (DDoS)

Risk

A DDoS attack is where the traffic of a targeted server or service is disrupted by overwhelming the infrastructure of the target with a flood of traffic [36]. DDoS attacks utilize bots (compromised computers) to send requests to the targets IP address with the intention of disrupting the targets service [36]. The impact of a DDoS attack on our application would cause resources of our hosting service to be exhausted which would result in downtime for the application, which could cause loss of business for the client.

Mitigation

In order to mitigate a DDoS attack on our application we need to implement several strategies. One strategy being rate limiting, which limits the number of requests a server will allow over a certain period of time [36]. Additionally implementing a Web Application Firewall (WAF) to filter the websites traffic based on rules [36]. Our application will be using Cloudflare WAF which includes DDoS protection, rate limiting, WAF and bot management [37]. Cloudflare WAF ensures enterprise grade security and availability, without having to create it from scratch allowing for faster development while staying protected.

Data Interception (Man-in-the-Middle)

Risk

A man-in-the-middle (MITM) attack is where an attacker positions themselves in between a user and an application with the intention of eavesdropping or impersonating either party [38]. The attacker's goal is to steal the user's personal information [38]. A risk of MITM attack is unencrypted user credentials or product data being intercepted over public networks.

Mitigation

In order to mitigate MITM attacks data in the application needs to be encrypted. This can be achieved by enforcing end-to-end TLS encryption using Cloudflare, while additionally setting strict HTTPS configurations on the hosting platform [39].

Brute Force and Credential Stuffing

Risk

Credential stuffing is where credentials which were obtained from a data breach are used to attempt a log in to an application or service unrelated to the credential's origin [40]. The attacker is hoping that the owner of the credentials has reused the same username and password for the different services [40]. Brute Force attacks are where the attacker attempts to gain access to a service without any clues or credentials from data breaches, they are essentially trying to gain access by providing common passwords [40]. The risk for our application would be for an attacker to use bots or to personally attempt to guess admin credentials with the attempt in gaining access to the admin portal.

Mitigation

This type of attack can be mitigated by implementing rate limiting which can prevent the number of times a bot can attempt to access an admin's account. Additionally, by implementing 2FA even with the correct credentials the attacker won't have access to an admin's account. Utilizing Cloudflare's bot management will ensure that malicious bot traffic does not reach our application preventing bot usage [41].

Authentication and Authorization (Admin Portal)

The admin portal makes use admin accounts to manage the public websites' catalogue and lead. The admin portal will make use of .NET Identity to secure and manage the created accounts. Identity ensures that the admins passwords are securely hashed and allows for 2FA to be enabled for extra account security. The use of 2FA ensures that if a password is compromised the account is still secure . Role-Based Access Control (RBAC) will also be present where there are three layers of access control. The "Super Admin" is the seeded account which has full permission for the application, allowing for admin accounts, products and leads to be managed. Below the "Super Admin" is the "Admin" which can only be assigned by the "Super Admin" and is an account meant for the owner of the business. This role has the same permissions as the "Super Admin" but excludes the ability to create other admin accounts. The final role in the admin portal is the "Employee" role, which allows for products and leads to be managed. The architecture of our application supports this through the Chain of Responsibility pattern, which implements authentication through the .NET Core middleware.

Network Security and Perimeter Defence

Secure Socket Layer (SSL) is a communication protocol which allows two devices to securely communicate with each other on a network [42]. SSL is an older communication protocol which contains security flaws [42]. Transport Layer Security (TLS) is an improved version of SSL which has fixed the previous security flaws [42]. TLS ensures that the data being transferred between two parties is encrypted and secure [43]. Additionally, TLS authenticates the exchanging parties ensuring they are who they say they are, while also verifying the integrity of the data [43]. Our application will make use of TLS to encrypt all traffic. The Cloudflare WAF will be our applications first layer of defence, where our applications host server will proxy through Cloudflare, which will ensure that the IP addresses of our docker containers remain hidden [44]. This ensures that all requests made to our application are optimized, caches and

protected [44]. Additionally, the WAF blocks bots, prevents DDoS attack and provides performance and security analytics [37].

Architectural Security

By implementing the Command Query Responsibility Segregation (CQRS) pattern all read and writes operations are separated which improves security as only the appropriate domain entity will have the permissions to perform write actions on the data being used [45]. Our frontend and backend projects will both be containerized using docker. Containers create an isolated environment, which means that if one container is compromised, the attackers cannot gain access to other containers [46]. Containerization reduces the total attack surface as containers do not have full OS-level virtualization [46]. Our data layer is made up of a PostgreSQL and Redis cache which is protected by a secure, authenticated connection from the backend which uses a singleton to create a single shared instance.

DevOps

DevOps is a development methodology that ensures both the development and operations teams are able to deliver software faster, securely and with more efficiency [47]. The DevOps methodology ensures that product delivery is accelerated through fast feedback and iterative improvement [47]. The ability to rapidly deploy a high-quality application is vital for the success of the Inflamm website and admin dashboard. DevOps supports remote collaboration which is crucial for our five-member development team in order to ensure that the applications we create are highly available and suffer zero-downtime from deployments.

Version Control and Branching Strategy

Version Control allows software development teams the ability to view a complete code history, and documentation for all file relevant to the development of the application [48]. Utilizing version control ensures every commit, review, experiment, comparison and reversal of code is entirely visible to the development team [48]. The Inflamm applications will make use of GitHub for version control, which will ensure that we have history of our codebase. GitHub will additionally improve our code quality where each developer will be encouraged to follow best practices to ensure that the code written is easy to access, understand, and maintain [48].

The project will be utilizing GitHub Flow workflow to implement a Feature Branching Strategy, which will ensure that our codebase is easy to maintain while receiving fewer merger conflicts [49].

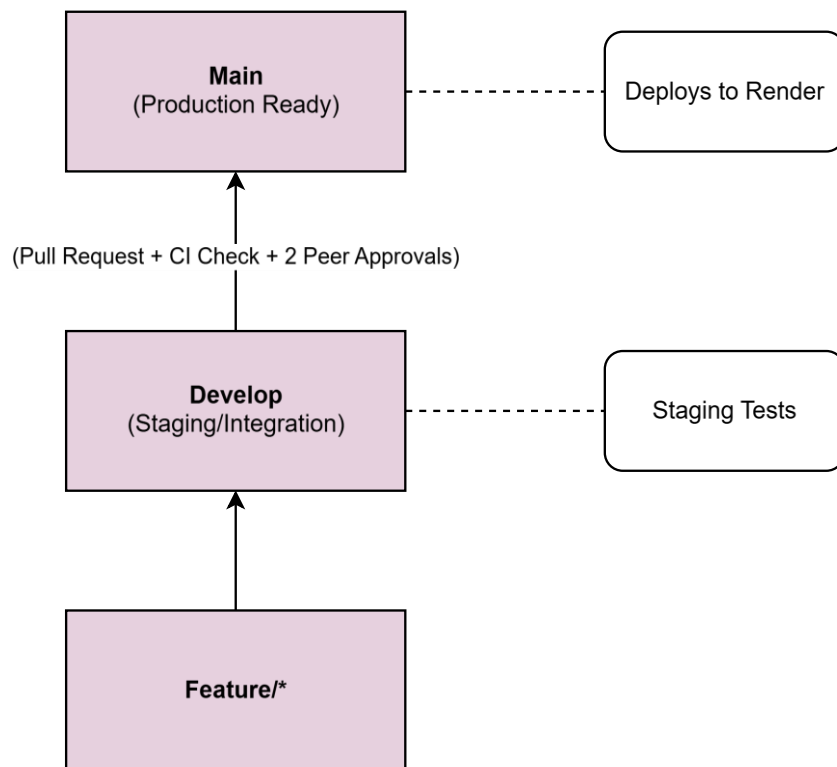


Figure 13: GitHub Flows Branch Structure

Branch Hierarchy

Main:

The main branch is for production ready code. All the code that is sitting in main is always stable and is an exact mirror of the containerized environment running live on Render. Main is to never be directly pushed to.

Develop:

The develop branch is where all feature branches are merged to after they have passed both automated testing and peer reviews.

Feature/*:

The feature branch is a short-lasting branch which is created off of the “develop” branch by each developer. The naming conventions for these branches all follow the same structure but differ depending on the task at hand (e.g. feature/admin-auth, feature/frontend-landing).

Collaboration and Pull Requests (PR) Workflow

Isolation

Each developer works on their assigned feature locally in either VS Code or Visual Studio.

Pull Request Submission

Once a developer has completed their assigned feature, they will submit a PR targeting the “develop” branch.

Code Review

A minimum of two team member are needed to review and approve each and every pull request. Once a PR is made it triggers the automated Continuous Integration (CI) pipeline, which requires all unit tests and integration to successfully pass before a merge is allowed.

Automated Test and Quality Assurance

Since Inflamm does not have a dedicated IT department, utilizing automated quality gates will ensure that broken code does not reach the live system, while ensuring faster delivery [50].

Backend Testing (C# .NET Core API)

Automated unit and integration tests will be executed using both xUnit and Moq. Where the tests will validate the CQRS query builders, Mediator command handlers, and the room calculation logic.

Frontend Testing (Next.js)

Testing the frontend will be done through utilizing the Jest and React testing library allowing us to test component rendering, user input validation, and states changes for both the public and admin portal.

Automated Quality Gate

Automated testing is integrated into the GitHub Actions CI process. This ensures that if any tests fail during a PR, GitHub actions will immediately block the pull request ensuring that only code that passes all test is able to reach production.

GitHub Actions Pipeline

The Continuous Integration (CI) / Continuous Deployment (CD) pipeline is responsible for automating the entire lifecycle of the codebase.

Phase 1: CI – Triggered on PR to “develop” or “main”

Environment Setup:

GitHub Actions supplies isolated Linux environments, where .NET Core SDK and Node.js environments are installed.

Dependency Restoration and Compilation:

NuGet packages and npm packages are restored, followed by a fully compilation.

Automated Test Execution:

xUnit and Jest are run which provide code coverage reports.

Static Security and Linting:

Scans codebase for dependencies vulnerabilities and security risks.

Phase 2: CD – Triggered on Merge to “main”

Container Building:

GitHub actions builds the Docker images for both applications.

Registry Push:

The Docker images are tagged with the commit hash which is then pushed to the GitHub Container Registry.

Deployment Webhook Execution:

The pipeline provides Render with an authenticated deploy webhook. This allows a zero-downtime container update for both hosted applications.

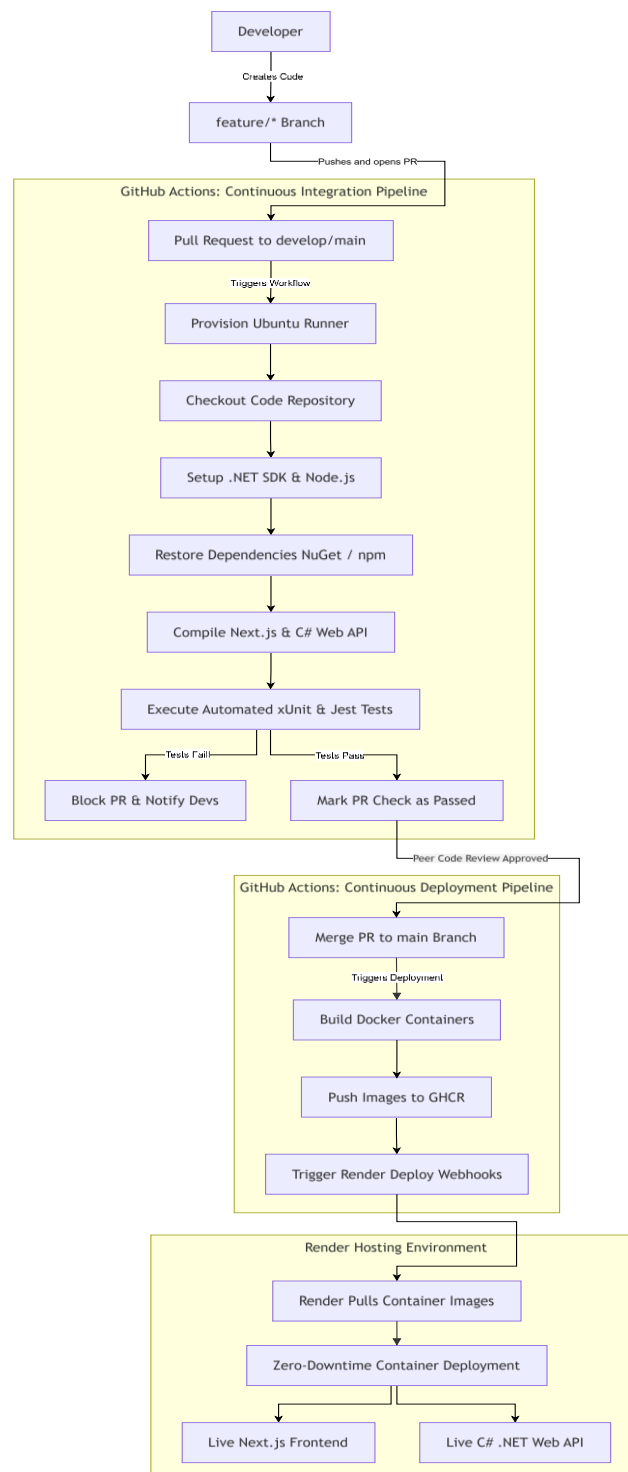


Figure 14: GitHub Actions Pipeline Diagram

Running Costs

Predicted Monthly Running Cost

The application is deployed as a fully managed service composed of cloud architecture. The C# .NET core Web API is packaged as a Docker container and hosted on Render, caching is provided by Upstash's serverless Redis platform, the PostgreSQL database and S3-compatible object storage are provided by Supabase and Cloudflare sits in front of the application as a free tier content delivery network and web application firewall.

Monthly Cost of service

Service	Purpose	Free tier limit	Paid tier
Render	API hosting (Docker container)	512 MB/0.1 CPU it spins down after a 15-minute idle	Starter \$7 monthly (512MB) Standard \$25 monthly (2 GB)
Supabase	PostgreSQL DB and image storage	500 MB DB, 1GB storage, 50K MAUs	Pro \$25 monthly and \$10 monthly compute credit (8 GB DB, 100GB storage)
Cloudflare	WAF, CDN, DDoS mitigation	Unmetered bandwidth baseline managed WAF	Pro \$20 monthly only if extra WAF rules are needed
Domain	DNS / Branding		\$12 -15 yearly or \$1.25 monthly
Transactional email	"Get quote" lead notifications to staff	3000 emails/monthly, 100 a day	Pro \$20 monthly if free daily cap is exceeded

Upstash	Redis product catalogue cache	256 MB, 500000 commands a month	Pay as you go: \$0.20/100000 commands and 0.25\$/GB storage over 1 GB
---------	-------------------------------	---------------------------------	---

Cost at scale

The total monthly infrastructure cost at three representative scales

Service	100 users monthly	1000 users monthly	10000 users monthly
Render	\$7	\$7-25	\$25-85 standard/Pro
Supabase	free	\$25 once free limits are exceeded	\$35-60
Upstash	free	\$0-5 pay as you go	\$15-35 pay as you go
Cloudflare	free	free	\$0 \$20 pro is optional
Domain	\$1.25	\$1.25	\$1.25
Email	free	\$0-5	\$10-20
Total	\$8-9 (ZAR129-145)	\$33-61 (ZAR533-985)	\$86-221 (ZAR1390-3572)

Two-Year Growth Projection

Period	Est. monthly users	Stack tier	Est. monthly cost
Months 1-3	50–100	All free tiers + Render Starter	\$8(ZAR129)
Months 4-6	300	Render Starter, Supabase Free	\$10(ZAR161)
Months 7-12	800–1,000	Supabase Pro triggered	\$40(ZAR646)
Year 2, Q1-Q1	2,500	Render Standard	\$60(ZAR969)
Year 2, Q3-Q4	5,000–10,000	Full paid stack	\$90–150(ZAR1454-2424)

Change Management

How and why will the organization implement the software?

The implementation of the new system is a responsibility of management and employees of Inflamm. Since the system replaces all existing outdated manual procedures and calculations of quotes via telephony or personally, records keeping about customers either using spread sheets or papers, and the inquiry procedure with the unified digital process, there is no need to convince users to implement it, because they will immediately feel all benefits of it. In particular, all actions are performed from the same staff dashboard (catalogue management, handling inquiries, calculation of quotes, and keeping the records about customers). Management retains oversight and control due to reporting and management permissions of the administrative accounts, which lowers potential risk of this process and guarantees organizational commitment. Moreover, since the time of calculations of quotes is significantly shortened, which means lower costs and higher sales capacity of Inflamm, there is a very strong financial reason for the ownership of this software system.

How and why will the users adopt the software?

The adoption of the software is the task of two groups of people, customers, and contractors/architects. The first group will find a solution to such issues like the lack of ability to visit shop in the work hours or to call to get answers on their questions. Customers will also be able to make an estimate quote, submit an inquiry, purchase small items 24/7 online from anywhere. Moreover, customers can use the recommendation calculator of heater size based on the dimension of the room, which will be a unique value of our shop. Contractors and architects will get the special workflow to receive faster the price quotes of custom multiple items.

Strategy to gain adoption from both the organisation and users

Strategy	Target Group	Purpose
Early inclusion of staff into UAT (user acceptance testing)	Staff/Admin	Establishes ownership and ensures the dashboard reflects real-world processes
Phased feature rollout (core features first, secondary features later)	Organisation	Mitigates risks and allows smooth adaptation of staff
Promotion of Customers quoting/purchasing within store and via email	Customers	Promotes early engagement and showcases convenience
Onboarding meeting and guide for Contractors/Architects	Contractors/Architects	Incentivizes the use of professional clients' ability to transition from telephone/email queries to a bulk quote process on the dashboard
Visible reporting of time/cost savings to management	Organisation	Reinforces continued executive support post-launch

Handover and Transition plan

The transition from development to production will take place via a carefully thought-out time-boxed roadmap minimizing disruption of Inflames day-to-day operations

Phase	Timeline	Activities
1. Pre-launch training	Weeks 1–2 before going live	Practical training sessions and delivery of the user guide to Staff and Admin about catalogue management, quoting, and customer record; onboarding meeting and guide to Contractors/Architects for bulk and customized quote request processing
2. Parallel running	Weeks 1–4 post-launch	Run of the new system simultaneously with old manual processes; cross-check of the quotes and customer records between the old processes and new system to detect inconsistencies early
3. Full cutover	End of Week 4	Official end of the old manual processes after confirmation of reliable operation and readiness of staff to use the new system
4. Intensive post-launch support	Months 1–3	Development provides priority support, including prompt responses for bugs influencing quotation and customer record accuracy
5. Long-term maintenance	Month 3 onward	Transition to a formal maintenance agreement (or internal IT contact) covering security patches, bug fixes, and minor feature updates

Additional training for Admin on system-wide reporting and account management is needed, as Admin users have extended rights compared to the usual Staff user account. The training material (guides, videos) will be provided after the go-live for

easy onboarding of new staff members without going through lengthy training sessions again.

Success Indicators

We will use the following indicators to judge success in the first three months after the adoption launch:

- **Use of system by staff:** 90% of the new quotations and customer record updates should be entered through the new system as compared to the manual way by the end of Month 1.
- **Customer interest:** We expect an increase in queries and quotes outside of regular shop working hours as evidence that the 24/7 service is used.
- **Uptake by Contractors/Architects:** At least 50% of bulk and custom quotations from Contractors are entered through the platform rather than by phone and email by the end of Month 2.
- **Declining number of support requests:** Support request rate goes down each month because users are comfortable with the new platform.

Contingency Plan

If the expected level of system adoption is not achieved, these contingency options will be used:

- **Extended parallel run:** The parallel run will be extended if there are still significant problems in the two systems by Week 4 in order to prevent loss of data and inconvenience to customers.

- **Additional training for user groups:** Additional training will be offered for particular user groups such as Contractors who do not seem to make enough use of the platform.
- Rollback option in case of critical system failure after a full cutover.

Prototype

There are three pages in the team project being:

1. Mobile Versions
2. Public Website Screens
3. Admin Website

Figma Prototype Link:

<https://www.figma.com/design/cHtB0Dv8BIfMgZAsVbUnRw/InFlame-Website?node-id=43-2426&t=2tIGwI03jWGhEsyX-1>

Declaration of authenticity

I, **Matthew Ethan Bowes** ID Number, **0501185133086** hereby declare that this portfolio, and any evidence included therein, contains my own independent work and that I have not received help from other groups.

I confirm that we have not committed plagiarism in the accomplishment of this work, nor have I falsified and/or invented experimental data.

I accept the academic penalties that may be imposed for violations of the above.

STUDENT NUMBER: **ST10444262**

STUDENT NAME: **Matthew Ethan Bowes**

STUDENT SIGNATURE: **Matthew Ethan Bowes** DATE: **2026/08/15**

I, **Leo Riley Addinal** ID Number, **0409305132089** hereby declare that this portfolio, and any evidence included therein, contains my own independent work and that I have not received help from other groups.

I confirm that we have not committed plagiarism in the accomplishment of this work, nor have I falsified and/or invented experimental data.

I accept the academic penalties that may be imposed for violations of the above.

STUDENT NUMBER: **ST10441154**

STUDENT NAME: **Leo Riley Addinal**

STUDENT SIGNATURE: **Leo Riley Addinal** DATE: **2026/08/15**

I, **Connor** ID Number, **0307045166088** hereby declare that this portfolio, and any evidence included therein, contains my own independent work and that I have not received help from other groups.

I confirm that we have not committed plagiarism in the accomplishment of this work, nor have I falsified and/or invented experimental data.

I accept the academic penalties that may be imposed for violations of the above.

STUDENT NUMBER: **ST10435598**

STUDENT NAME: **Connor Kelly McClure**

STUDENT SIGNATURE: **Connor Kelly McClure** DATE: **2026/08/15**

I, **Nafees Sampson** ID Number, **0312155335085** hereby declare that this portfolio, and any evidence included therein, contains my own independent work and that I have not received help from other groups.

I confirm that we have not committed plagiarism in the accomplishment of this work, nor have I falsified and/or invented experimental data.

I accept the academic penalties that may be imposed for violations of the above.

STUDENT NUMBER: **ST10456389**

STUDENT NAME: **Nafees Sampson**

STUDENT SIGNATURE: **Nafees Sampson** DATE: **2026/08/15**

I, **Thomas Newman** ID Number, **0506085427082** hereby declare that this portfolio, and any evidence included therein, contains my own independent work and that I have not received help from other groups.

I confirm that we have not committed plagiarism in the accomplishment of this work, nor have I falsified and/or invented experimental data.

I accept the academic penalties that may be imposed for violations of the above.

STUDENT NUMBER: **ST10456111**

STUDENT NAME: **Thomas Newman**

STUDENT SIGNATURE: **Thomas Newman** DATE: **2026/08/15**

References

- [1 Refactoring Guru, "Singleton," Refactoring Guru, 2026. [Online]. Available:
] <https://refactoring.guru/design-patterns/singleton>. [Accessed 2026].
- [2 Redis, "Connection pools and multiplexing," Redis, 2026. [Online]. Available:
] <https://redis.io/docs/latest/develop/clients/pools-and-muxing/>. [Accessed 2026].
- [3 Supabase, "C# Client Library," Supabase, 2026. [Online]. Available:
] <https://supabase.com/docs/reference/csharp/introduction>. [Accessed 2026].
- [4 Microsoft, ".NET dependency injection," Microsoft, 2026. [Online]. Available:
] <https://learn.microsoft.com/en-us/dotnet/core/extensions/dependency-injection/overview>.
[Accessed 2026].
- [5 Project Management Institute, "The Singleton Pattern," Project Management Institute, 2026.
] [Online]. Available: <https://www.pmi.org/disciplined-agile/the-design-patterns-repository/the-singleton-pattern>. [Accessed 2026].
- [6 Refactoring Guru, "Factory Method," Refactoring Guru, 2026. [Online]. Available:
] <https://refactoring.guru/design-patterns/factory-method>. [Accessed 2026].
- [7 Refactoring Guru, "Builder," Refactoring Guru, 2026. [Online]. Available:
] <https://refactoring.guru/design-patterns/builder>. [Accessed 2026].
- [8 S. Kumar, "Builder Design Pattern," GeeksForGeeks, 6 May 2026. [Online]. Available:
] <https://www.geeksforgeeks.org/system-design/builder-design-pattern/>. [Accessed 2026].
- [9 O. Koroglu, ".NET Predicate Builder Pattern: A Powerful Method for Dynamic Queries," Medium,
] 25 March 2025. [Online]. Available: <https://medium.com/@koroglu1795/net-predicate-builder-pattern-a-powerful-method-for-dynamic-queries-6ea8274b8027>. [Accessed 2026].
- [1 Refactoring Guru, "Adapter," Refactoring Guru, 2026. [Online]. Available:
0] <https://refactoring.guru/design-patterns/adapter>. [Accessed 2026].
- [1 Refactoring Guru, "Decorator," Refactoring Guru, 2026. [Online]. Available:
1] <https://refactoring.guru/design-patterns/decorator>. [Accessed 2026].
- [1 Refactoring Guru, "Facade," Refactoring Guru, 2026. [Online]. Available:
2] <https://refactoring.guru/design-patterns/facade>. [Accessed 2026].
- [1 Refactoring Guru, "Command," Refactoring Guru, 2026. [Online]. Available:
3] <https://refactoring.guru/design-patterns/command>. [Accessed 2026].
- [1 Microsoft, "Command Query Responsibility Segregation (CQRS) pattern," Microsoft, 2025.
4] [Online]. Available: <https://learn.microsoft.com/en-us/azure/architecture/patterns/cqrs>. [Accessed 2025].
- [1 Refactoring Guru, "Mediator," Refactoring Guru, 2026. [Online]. Available:
5] <https://refactoring.guru/design-patterns/mediator>. [Accessed 2026].
- [1 V. Desai, "Implementing CQRS And Mediator Patterns With ASP.NET Core Web API," C#
6] Corner, 2024. [Online]. Available: <https://www.c-sharpcorner.com/article/implementing-cqrs-and-mediator-patterns-with-asp-net-core-web-api/>. [Accessed 2026].
- [1 Refactoring Guru, "Observer," Refactoring Guru, 2026. [Online]. Available:
7] <https://refactoring.guru/design-patterns/observer>. [Accessed 2026].

- [1 Refactoring Guru, "State," Refactoring Guru, 2026. [Online]. Available:
8] <https://refactoring.guru/design-patterns/state>. [Accessed 2026].
- [1 Refactoring Guru, "Chain of Responsibility," Refactoring Guru, 2026. [Online]. Available:
9] <https://refactoring.guru/design-patterns/chain-of-responsibility>. [Accessed 2026].
- [2 Microsoft, "ASP.NET Core middleware," Microsoft, 2026. [Online]. Available:
0] <https://learn.microsoft.com/en-us/aspnet/core/fundamentals/middleware/?view=aspnetcore-10.0>. [Accessed 2026].
- [2 M. Tanner, "Enterprise software architecture patterns: The complete guide," vFunction, 2025.
1] [Online]. Available: <https://vfunction.com/blog/enterprise-software-architecture-patterns/>. [Accessed 2025].
- [2 tecnovy, "Top 10 Software Architecture & Design Patterns of 2026," tecnovy, 2026. [Online].
2] Available: <https://tecnovy.com/en/top-10-software-architecture-patterns>. [Accessed 2026].
- [2 V. Walker, "14 software architecture design patterns to know," Red Hat, 2022. [Online]. Available:
3] <https://www.redhat.com/en/blog/14-software-architecture-patterns>. [Accessed 2022].
- [2 Postman, "Guide to API-first," Postman, 2026. [Online]. Available: <https://www.postman.com/api-4-first/>. [Accessed 2026].
- [2 supabase , "Architecture," supabase, 2026. [Online]. Available:
5] <https://supabase.com/docs/guides/getting-started/architecture>. [Accessed 2026].
- [2 Docker, "What is Docker?," Docker, 2026. [Online]. Available: <https://docs.docker.com/get-started/docker-overview/>. [Accessed 2026].
- [2 Render, "Render Documentation," Render, 2026. [Online]. Available: <https://render.com/docs>.
7] [Accessed 2026].
- [2 upstash, "Getting Started," upstash, 2026. [Online]. Available:
8] <https://upstash.com/docs/redis/overall/getstarted>. [Accessed 2026].
- [2 Cloudflare, "Reference Architectures," Cloudflare, 2026. [Online]. Available:
9] https://developers.cloudflare.com/reference-architecture/?_gl=1*11pjswr*_gcl_au*NjkkxNjA0Mzg5LjE3ODU5MTc3NzM.*_ga*NmQwNjdINjUtOVVmYS00OTJiLWE3MmltNWZmOWYyM2I5ZmI5. [Accessed 2026].
- [3 Redis, "Redis serialization protocol specification," Redis, 2026. [Online]. Available:
0] <https://redis.io/docs/latest/develop/reference/protocol-spec/>. [Accessed 2026].
- [3 PostgreSQL, "18.9. Secure TCP/IP Connections with SSL," PostgreSQL, 2026. [Online].
1] Available: <https://www.postgresql.org/docs/current/ssl-tcp.html>. [Accessed 2026].
- [3 PortSwigger, "Cross-site scripting," PortSwigger, 2026. [Online]. Available:
2] <https://portswigger.net/web-security/cross-site-scripting>. [Accessed 2026].
- [3 PortSwigger, "Content security policy," PortSwigger, 2026. [Online]. Available:
3] <https://portswigger.net/web-security/cross-site-scripting/content-security-policy>. [Accessed 2026].
- [3 L. Hallie, "Understanding XSS Attacks," Vercel, 2025. [Online]. Available:
4] <https://vercel.com/kb/guide/understanding-xss-attacks>. [Accessed 2025].
- [3 PortSwigger, "Access control vulnerabilities and privilege escalation," PortSwigger, 2026.
5] [Online]. Available: <https://portswigger.net/web-security/access-control>. [Accessed 2026].

- [3] Cloudflare, "What is a DDoS attack?," Cloudflare, 2026. [Online]. Available:
6] <https://www.cloudflare.com/learning/ddos/what-is-a-ddos-attack/>. [Accessed 2026].
- [3] Cloudflare, "WAF," Cloudflare, 2026. [Online]. Available:
7] <https://www.cloudflare.com/products/waf/>. [Accessed 2026].
- [3] Imperva, "What is MITM attack," Imperva, 2026. [Online]. Available:
8] <https://www.imperva.com/learn/application-security/man-in-the-middle-attack-mitm/>. [Accessed 2026].
- [3] SentinelOne, "How to Prevent MitM Attacks?," SentinelOne, 2026. [Online]. Available:
9] <https://www.sentinelone.com/cybersecurity-101/threat-intelligence/how-to-prevent-mitm-attacks/>. [Accessed 2026].
- [4] Cloudflare, "What is credential stuffing? | Credential stuffing vs. brute force attacks," Cloudflare, 2026. [Online]. Available: <https://www.cloudflare.com/th-th/learning/bots/what-is-credential-stuffing/>. [Accessed 2026].
- [4] Cloudflare, "Bot Mitigation," Cloudflare, 2026. [Online]. Available:
1] <https://www.cloudflare.com/products/bot-mitigation/>. [Accessed 2026].
- [4] AWS, "What's the Difference Between SSL and TLS?," AWS, 2026. [Online]. Available:
2] <https://aws.amazon.com/compare/the-difference-between-ssl-and-tls/>. [Accessed 2026].
- [4] Cloudflare, "What is TLS (Transport Layer Security)?," Cloudflare, 2026. [Online]. Available:
3] <https://www.cloudflare.com/learning/ssl/transport-layer-security-tls/>. [Accessed 2026].
- [4] Cloudflare, "Proxy status," Cloudflare, 2026. [Online]. Available:
4] <https://developers.cloudflare.com/dns/proxy-status/>. [Accessed 2026].
- [4] Microsoft, "Command Query Responsibility Segregation (CQRS) pattern," Microsoft, 2025.
5] [Online]. Available: <https://learn.microsoft.com/en-us/azure/architecture/patterns/cqrs>. [Accessed 2025].
- [4] B. Buckman, "Containerization in Cybersecurity Explained," Huntress, 2025. [Online]. Available:
6] <https://www.huntress.com/cybersecurity-101/topic/what-is-containerization-cybersecurity-guide>. [Accessed 2025].
- [4] GitLab, "What is DevOps?," GitLab, 2026. [Online]. Available:
7] <https://about.gitlab.com/topics/devops/>. [Accessed 2026].
- [4] GitHub, "What is Version Control?," GitHub, 2024. [Online]. Available:
8] <https://github.com/resources/articles/what-is-version-control>. [Accessed 2024].
- [4] Harness, "Github Flow vs. Git Flow: What's the Difference?," Harness, 2023. [Online]. Available:
9] <https://www.harness.io/blog/github-flow-vs-git-flow-whats-the-difference>. [Accessed 2023].
- [5] M. Shah, "Autonomous Quality Gates: AI-Powered Code Review," Augment Code, 2025. [Online].
0] Available: <https://www.augmentcode.com/learn/autonomous-quality-gates-ai-powered-code-review>. [Accessed 2025].